

How OmniReads recommends books

A walk-through of the recommendation system you built: what each step does, why, and where it lives in the code. Every number and example below was computed on your real data.

The idea in one sentence

OmniReads turns every book into a list of **384 numbers** that capture its meaning. When you like a book, it looks for the books whose numbers are **closest** to it. When you dislike one, it pushes away the books close to it. This family of methods is called **content-based filtering**: recommendations come from what the books are *about*, not from what other people did.

The two phases

The system is split in two. The heavy work (reading 20,000 descriptions with an AI model) is done **once, offline**. Its result is saved in two files. The website then only does cheap arithmetic on those files, **online**, every time someone opens the page.

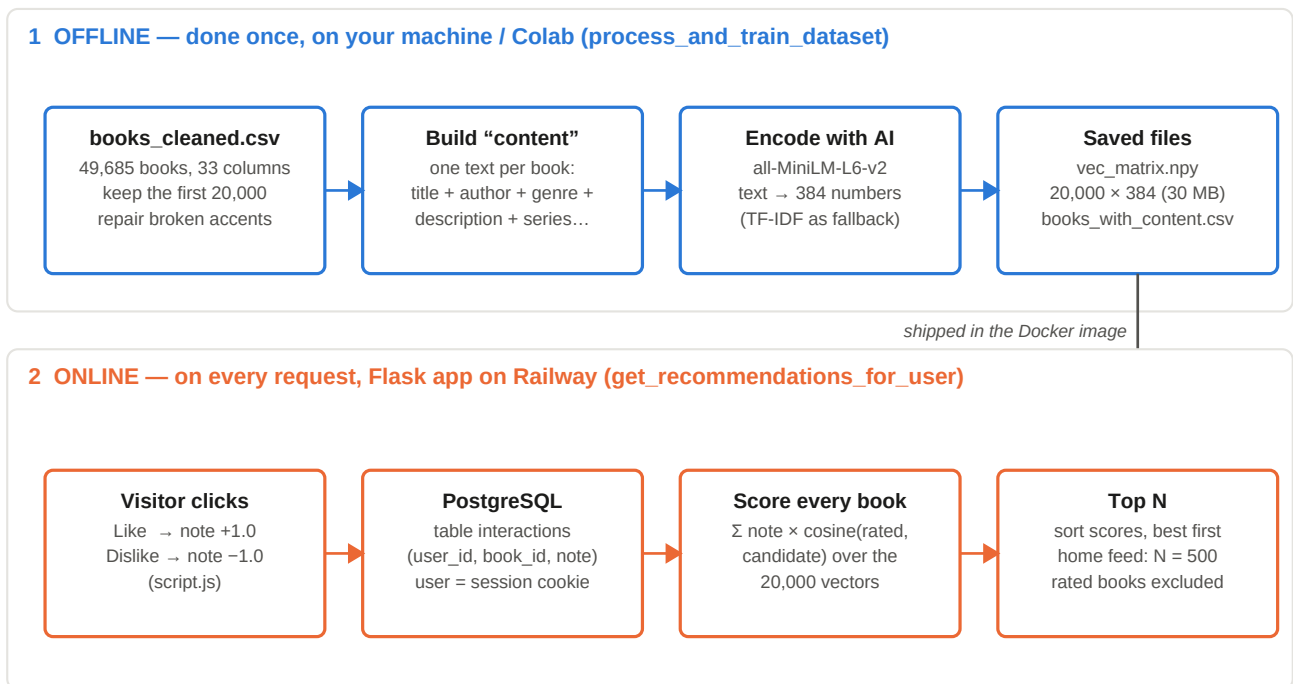


Figure 1 — The offline phase produces the vectors once; the online phase reuses them on every request.

Why split it like this?

Encoding 20,000 books with the AI model takes several minutes and needs PyTorch (several GB). Comparing numbers takes milliseconds and needs only NumPy. So the deployed app never runs the AI model: that's exactly why we could remove *sentence-transformers* from the production requirements.

Step 1 — Prepare the data

The raw file **books_cleaned.csv** holds **49,685 books** with 33 columns (title, author, description, genres, ratings, pages, awards...). Training keeps the **first 20,000 rows** (`model.py:188, .head(20000)`) to keep memory and encoding time reasonable.

Many texts had **broken accents**, a problem called *mojibake*: “é” saved in one encoding and read back in another shows up as “Ã©”. `sanitize_text()` repairs this by re-encoding the text as Latin-1 and decoding it back as UTF-8, which reverses the mistake.

Step 2 — Give each book one “content” text

The model reads one piece of text per book. So you glue the useful columns together into a single field called **content**. The trick: some fields are **written twice**. The more often a word appears, the more it shapes the meaning the model sees, so repeating a field is a simple way to make it count more.

Field	Times	Why
title	1×	Identifies the book
author	2×	Readers often want more by the same author
description	1×	Already long: it dominates naturally
main_genre	2×	Strongest signal of taste (Fantasy vs Romance)
second_genre	1×	Refines the genre
series	2×	Books of the same series should stick together
publisher	1×	Weak hint (imprints often specialise)

Real example, the end of the content of *The Book of Lost Tales, Part Two*. After the description come the main genre twice, the second genre, the series twice and the publisher:

```
...previously unpublished writings of J.R.R. Tolkien."
Fantasy Fantasy Fiction (The History of Middle-Earth) (The History of Middle-Earth) Del Rey
```

Step 3 — Turn text into numbers

A computer can't compare sentences, but it can compare lists of numbers (**vectors**). Your notebook shows you tried two ways to do the conversion.

First attempt: TF-IDF

TF-IDF gives each **word** a score: high if the word is frequent *in this book* (TF, term frequency) and rare *in the whole collection* (IDF, inverse document frequency). “Hobbit” scores high; “the” scores zero (it's removed as a stop word anyway). Each book becomes a vector with one slot per word of the vocabulary.

Its weakness: it only matches **identical words**. A book about a “sorcerer” and one about a “wizard” look unrelated, because the words differ.

Final choice: Sentence Transformers

You switched to **all-MiniLM-L6-v2**, a small neural network trained on millions of sentence pairs to put texts with similar **meaning** close together, even when they share no words. It reads each content text and outputs **384 numbers** (an *embedding*). The 384 slots don't have human names like “fantasy level”; the meaning is spread across all of them.

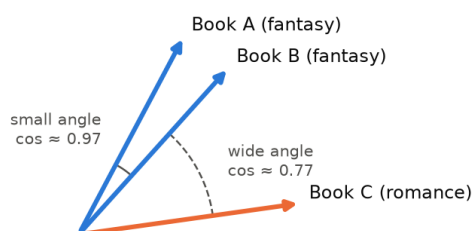
	TF-IDF	Sentence Transformers (used)
Compares	exact words	meaning
Vector size	one slot per word (tens of thousands)	384, fixed
Wizard $\approx$ sorcerer?	no	yes
Cost	seconds, scikit-learn only	minutes, needs PyTorch
In the code	fallback if the model fails (model.py:215)	model.py:209

The result is saved in **vec_matrix.npy**: a table of **20,000 rows $\times$ 384 columns** (30 MB). Row i is the vector of the book on row i of **books_with_content.csv**. That's how the app finds a book's vector: it looks up the book's row number, then reads the same row in the matrix.

Step 4 — Measure how similar two books are

Picture each vector as an arrow from the origin. Books about similar things point in **similar directions**. **Cosine similarity** measures the angle between two arrows:

$$\cos(A, B) = (A \cdot B) / (|A| \times |B|)$$



$A \cdot B$ (dot product) multiplies the 384 pairs of numbers and adds them up.

$|A|$ is the length of the arrow. Dividing by it means only the *direction* counts, not how long the description was.

Your vectors already have length 1 (checked: 0.99999994 to 1.0), so the cosine here is just the dot product.

Figure 2 — Same direction: cosine near 1. Unrelated: near 0. Opposite: negative.

What do real scores look like? Here is the similarity between *The Book of Lost Tales, Part Two* (Tolkien) and each of the 19,999 other books:

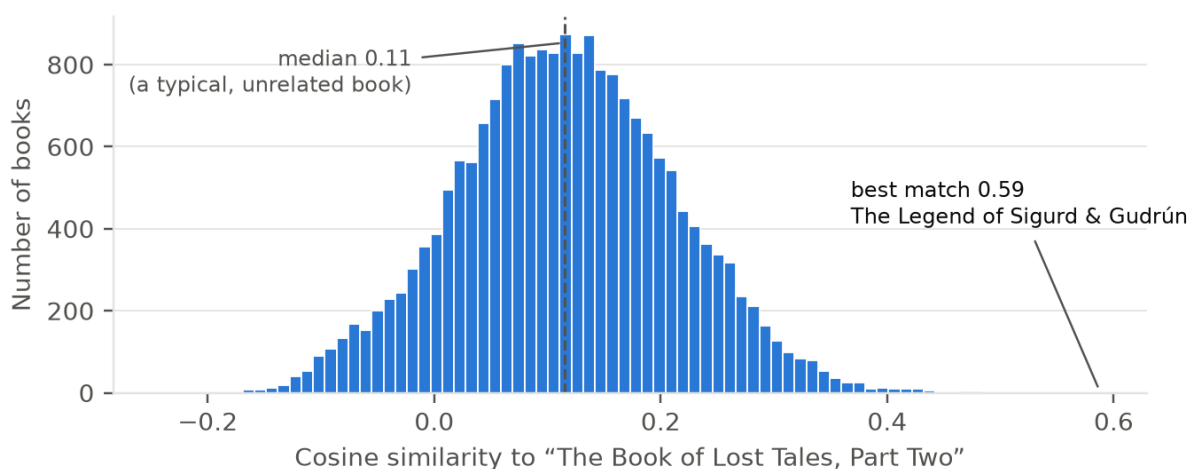


Figure 3 — Most books score around 0.11 against a Tolkien book. Only 1% score above 0.34. The top match, at 0.59, is another Tolkien book.

Reading the scores

Don't expect values near 1. With this model, **0.5 is already a very close match** and ~0.1 means "unrelated". What matters is the **ranking**, not the absolute number.

Step 5 — From your likes to a ranked list

This is the heart of the system, in `get_recommendations_for_user` (`model.py:266`). Every book b of the catalogue gets a score:

$$\text{score}(b) = \sum_{\text{rated books } r} \text{note}(r) \times \cos(r, b)$$

In words: for each book you rated, take its similarity to b , multiply by your note (+1 for a like, -1 for a dislike), and add everything up. Then:

1. Books you already rated get **-10,000** so they never come back.
2. Sort from highest to lowest score.
3. Return the first N (500 for the home feed, `app.py:89`).

```
score = np.zeros(len(books))          # one score per book, all 0 at the start
for book_id, note in interactions.items(): # each like/dislike of this user
    realindex = books[books['id'] == book_id].index[0] # its row number
    similarity = cosine_similarity(vec_matrix[realindex].reshape(1, -1),
                                  vec_matrix).flatten() # vs. all 20,000 books
    score += similarity * note # like pulls closer, dislike pushes away
    score[realindex] = -10000 # never recommend a rated book
# then: sort by score, keep the top n
```

Example A — one like

You like *The Book of Lost Tales, Part Two* (Tolkien, Fantasy). The score of each book is simply its similarity to that one book:

#	Recommended book	Author	Score
1	The Legend of Sigurd & Gudrún	J.R.R. Tolkien	0.588
2	Roverandom	J.R.R. Tolkien	0.544
3	In the Land of Time: And Other Fantasy Tales	Lord Dunsany	0.515
4	The Monsters and the Critics and Other Essays	J.R.R. Tolkien	0.509
5	The Written	Ben Galley	0.492
6	The Fall of Arthur	J.R.R. Tolkien	0.486

Four Tolkien books (helped by the doubled author field) plus two other fantasy books.

Example B — two different likes

Now you also like *Pride and Prejudice* and *Zombies: The Graphic Novel*. Each book's score is the **sum** of its similarity to both:

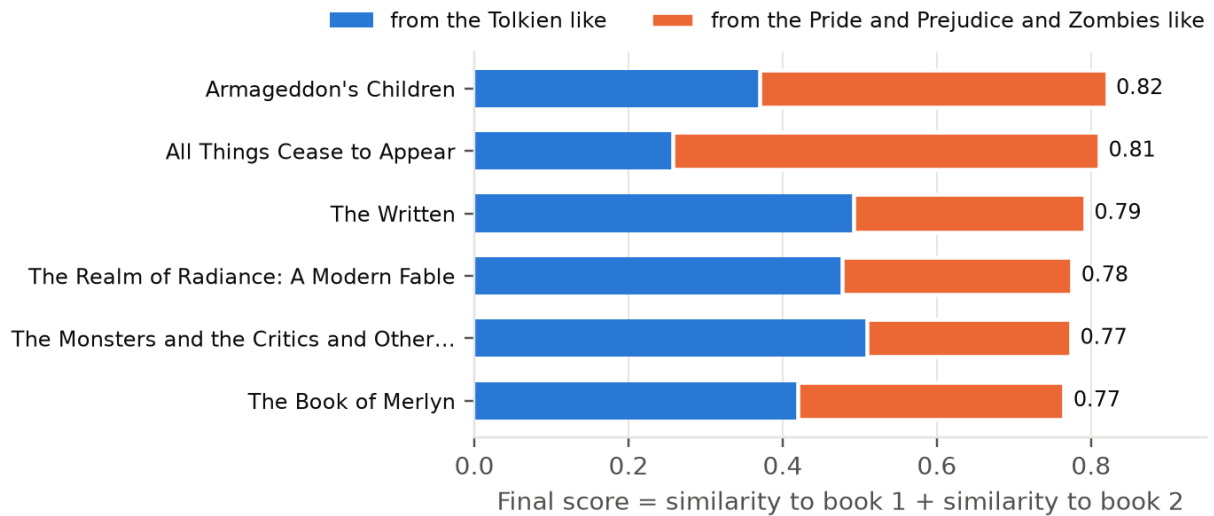


Figure 4 — Each bar is split into the part coming from each like. The winners are books moderately close to both, rather than very close to just one.

The Legend of Sigurd ($0.59 + 0.11 = 0.70$) drops out of the top 6, beaten by *Armageddon's Children* ($0.37 + 0.45 = 0.82$). Adding similarities rewards **compromise** books. That's good for variety, but it can surface odd middle grounds like *All Things Cease to Appear*, a literary thriller.

Example C — a like and a dislike

You like the Tolkien book but dislike *Roverandom*, another Tolkien book:

#	Recommended book	cos with liked	cos with disliked	Score
1	How the Irish Saved Civilization	0.320	0.052	0.268
2	Island of Graves	0.292	0.058	0.234
3	Island of Silence	0.302	0.068	0.234
4	Two Years Before the Mast	0.223	-0.007	0.229
5	Longitude	0.323	0.098	0.224

Lesson: a dislike weighs as much as a like

Every other Tolkien book is close to *both* books, so the like and the dislike cancel out: for *The Legend of Sigurd*, $+0.59 - 0.45 = 0.14$, now below books unrelated to Tolkien (0.27). All of Tolkien disappears from the top. One dislike erased an entire author, because the two books share so much.

When you haven't rated anything yet

With no ratings, there's nothing to compare, the classic **cold start** problem. The code returns the first N books of the CSV (model.py:287): *Inner Circle*, *A Time to Embrace*, *Take Two*... This is just file order, not popularity.

What about collaborative filtering?

Your notebook went further. It explored **user-based collaborative filtering**: “people with similar tastes in the past will like similar books in the future”. You built a **user × book matrix** from 30 simulated users (7,428 interactions on 2,908 books), compared users with cosine similarity, and recommended what similar users liked. Then you designed a **hybrid** combining both approaches, since each covers the other's blind spots:

Situation	Content-based	User-based
New book with no ratings yet	works	invisible
User with few ratings	works	weak similarity
Discovering outside your usual genres	limited	works

In the deployed app, only content-based is active

`model.py:305-311` builds the user × book matrix and computes `user_sims`, but that variable is **never used** afterwards: the final ranking comes only from the content score. So the live app is 100% content-based. That's actually reasonable today: collaborative filtering needs many real users before it says anything useful.

Where each piece lives in the app

Endpoint	What it does	Uses the AI vectors?
GET /api/home	Top 500 recommendations for the visitor	yes
GET /api/recommend?n=10	Same thing, N chosen by the caller	yes
POST /api/interact	Saves a like (+1) or dislike (-1) in PostgreSQL	—
POST /api/remove_interact	Removes a rating	—
GET /api/my_ratings	Your rated books (“My Library”)	no
GET /api/search?q=...	Keyword match on title + author + genre, max 80	no (plain text)
GET /api/genres	Top 10 genres, 15 books each	no
POST /api/train	Runs the offline phase if the files are missing	creates them

Ratings are tied to a visitor through a signed session cookie that holds a random ID; the ratings themselves live in PostgreSQL (tables `users` and `interactions`).

Limits and ideas to improve it

Limit today	Idea
A dislike cancels a like with equal force (Example C)	Give dislikes a smaller weight, e.g. -0.5
Cold start shows books in file order	Show the most popular books instead: the raw CSV has <i>rating_count</i> and <i>average_rating</i>
Only 20,000 of 49,685 books are used	Encode the full set: ~75 MB of vectors, still small
Search is keyword-only: "wizard school" won't find Harry Potter	Semantic search: encode the query with the same model and rank by cosine (needs the model at runtime, or a lighter ONNX version)
Collaborative filtering is computed but unused	Once there are real users, blend: $\text{final} = \alpha \times \text{content} + (1 - \alpha) \times \text{collaborative}$
The top results can be near-duplicates (same series)	Diversify the list, e.g. with Maximal Marginal Relevance (MMR)

Glossary

- **Vector / embedding** — A list of numbers representing a piece of text. Here, 384 numbers per book.
- **Cosine similarity** — Score between -1 and 1 measuring whether two vectors point the same way.
- **Content-based filtering** — Recommends items similar to the ones you liked, based on their description.
- **Collaborative filtering** — Recommends what users similar to you liked, based on behaviour only.
- **Cold start** — The problem of recommending to a new user (or a new item) with no history.
- **TF-IDF** — Word-scoring method: frequent in this document $\times$ rare in the collection.
- **Sentence Transformer** — Neural network that turns a sentence into an embedding capturing its meaning.
- **Mojibake** — Garbled text caused by decoding characters with the wrong encoding ("Ã©" instead of "é").